12 11 2026

Executive Report – Project III: AI Video Knowledge Assistant

1. Project Overview

The AI Video Knowledge Assistant is a web application developed to make information contained in YouTube videos easier to access.

The idea behind the project is straightforward: users often need to rewatch long videos to find a particular explanation, recommendation, or piece of information. This application allows them to ask questions in natural language and receive written answers based on the content of a selected video's transcript.

The intended audience is the general public, particularly people interested in health, wellbeing, exercise, and the benefits of physical activity. The application is not designed exclusively for athletes or sports professionals.

The current catalogue contains nine English-language videos.

2. Main Features

The application includes the following functionality:

- **Video-based question answering:** users can ask questions about the content of a selected video.
- **Voice input:** users can record a question instead of typing it. The audio is sent to the backend for transcription before being processed.
- **Transcript retrieval:** the system searches for relevant passages rather than relying only on the language model's general knowledge.
- **Conversational interaction:** users can ask multiple questions about the same video and continue the conversation.
- **Video summaries:** the agent can generate summaries of the selected video's content.
- **References and timestamps:** retrieved passages retain timestamp information to help users locate the relevant section in the original video.

3. Technical Implementation

The project combines a web frontend, a Python backend, an LLM-based agent, and a vector database.

Frontend

- Next.js
- React
- TypeScript

Backend

- Python
- FastAPI
- LangChain
- LangGraph

AI and data

- OpenAI GPT-5.4-mini as the default language model
- OpenAI text-embedding-3-small for embeddings
- Neon PostgreSQL with pgvector for the deployed retrieval workflow
- SQLite for LangGraph conversation checkpoints
- ChromaDB for the catalogue preparation and indexing script

The frontend is deployed on Vercel, while the backend runs on Render. Neon provides the PostgreSQL database used by the retrieval system.

4. Data Processing and Retrieval

The application works with a predefined catalogue of nine videos.

During catalogue preparation, the system retrieves English-language YouTube transcripts, cleans the transcript segments, and divides the text into overlapping chunks.

The current chunking configuration uses:

- Chunk size: 1,000 characters
- Overlap: 150 characters

Timestamp and segment information are retained as metadata.

For the deployed application's question-answering workflow, transcript chunks are retrieved from Neon PostgreSQL using pgvector. The user's question is converted into an embedding and

compared with stored transcript embeddings. Relevant passages are then supplied to the AI agent as context for generating an answer.

ChromaDB is also present in the codebase and is used by the catalogue-building script. It is important to distinguish this preparation workflow from the Neon-based retrieval used by the deployed application.

5. AI Agent and Conversation Management

The conversational component is implemented with LangChain and LangGraph.

The agent has access to three dedicated tools:

1. search_transcript – retrieves transcript passages relevant to the user's question.
2. summarize_current_video – generates a summary of the selected video.
3. retrieve_video_references – retrieves relevant transcript references.

The agent is instructed to use transcript evidence when answering factual questions about a video and to distinguish information found in the video from general knowledge.

Conversation checkpoints are managed through LangGraph's SQLite checkpointer. The conversation identifier and video identifier are used together to maintain the relevant context.

6. Testing and Evaluation

Testing focused on the main user workflows and the interaction between the frontend, backend, retrieval layer, and AI agent.

The application was tested through the deployed interface, including:

- Selecting and querying videos from the available catalogue.
- Asking multiple questions about the same video.
- Asking questions concerning different sections of a transcript.
- Using voice recording to submit questions.
- Checking whether retrieved passages were relevant to the question.

The project also includes evaluation scripts and LangSmith integration for tracing and inspecting retrieval and answer-generation behaviour.

A retrieval test using the question *"What are some of Karen's favorite recovery strategies?"* returned three relevant transcript passages. All three were judged relevant in that specific test. This is an individual test result, not an overall accuracy measurement.

No general accuracy percentage is reported because a sufficiently broad and systematically validated evaluation has not been established.

7. Deployment and Current Limitations

The application is publicly accessible:

- **Frontend:** https://ai-video-knowledge-assistant.vercel.app/
- **Backend:** https://ai-video-knowledge-assistant.onrender.com

The current implementation has some defined limitations:

- It supports a predefined catalogue of nine English-language videos.
- Users cannot freely submit any YouTube URL for ingestion through the deployed interface.
- The system works with transcript content and does not directly analyse video frames or visual information.
- Answer quality depends on transcript availability and the relevance of the retrieved passages.
- Further systematic evaluation would be required to establish reliable quantitative performance metrics.

8. Final Status

The application is deployed and its main interaction flows have been tested.

The source code, updated README documentation, environment-variable example, dependency list, and Git configuration have been published to GitHub.

Repository: https://github.com/ivanmatteuzzi/AI-video-knowledge-assistant

The project demonstrates the practical integration of LLMs, retrieval-augmented generation, vector search, conversational agents, speech input, persistent conversation context, and cloud deployment in a working application.

The main result is a functional system that makes information from a small, predefined collection of educational videos accessible through written and spoken questions.